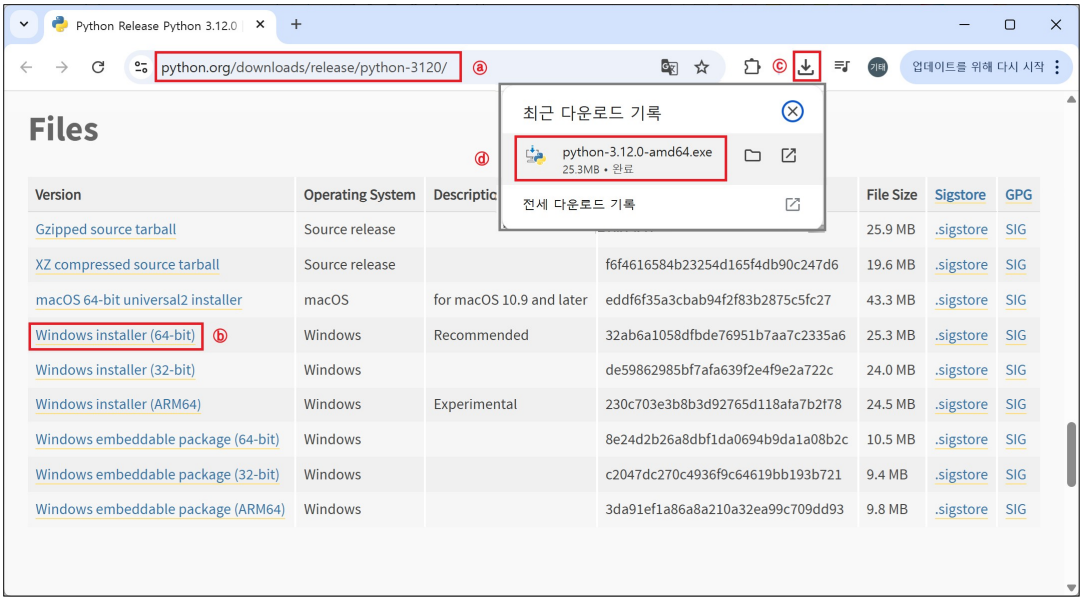
1. Python 개발 환경 구성 및 가상환경 설정

이 절에서는 FastAPI 개발을 위한 Python 설치 및 환경 설정 방법을 학습한다. Python은 성능 향상과 타입 힌트 강화로 FastAPI와의 궁합이 탁월하다. 최신 버전을 설치하고, 프로젝트별 독립 실행 환경을 구성하기 위해 venv 가상환경을 사용하는 방법을 익힌다. 또한 Windows 명령어를 확인하고, 올바른 경로 설정과 환경 변수 등록 절차를 실습을 통해 명확히 이해한다.

(1) Python 다운로드 및 설치

FastAPI의 개발 환경에 앞서 일단, Python을 먼저 설치해야 합니다. 여기에서는 Python 3.12의 설치 파일을 다운로드 받아 설치하면, 고성능으로 데이터 과학, 머신러닝, AI 모델 학습에 안정적인 환경을 제공한다. Windows 환경에서 공식 Python 사이트를 통해 설치하며, PATH 환경 변수 등록을 통해 터미널에서 바로 Python을 실행할 수 있도록 설정한다.

① 파이썬을 다운받는 사이트(https://www.python.org/downloads/release/python-3120)에 방문하여 [Windows Installer (64bit)] 링크를 눌러 다운로드를 받는다.



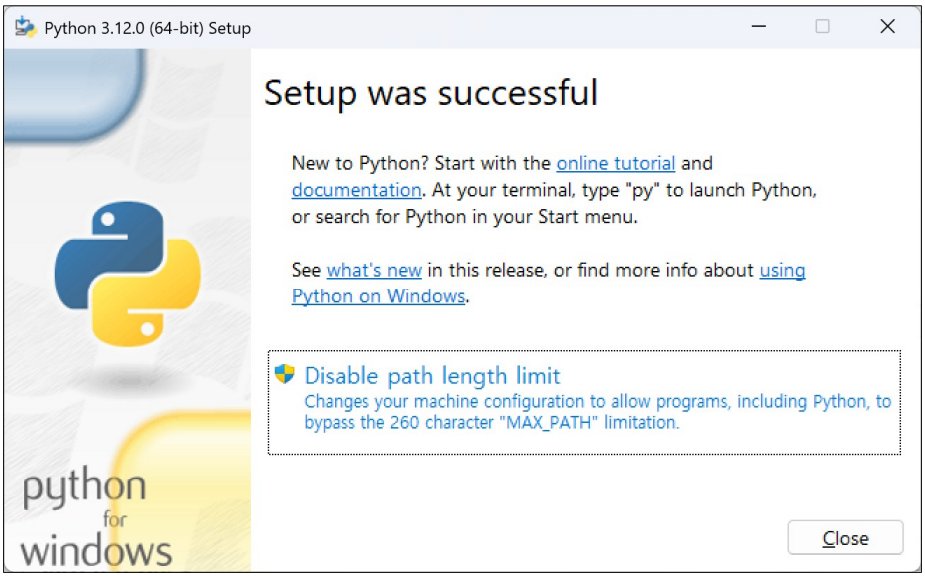
[그림 1] 파이썬 다운로드 사이트

② 다운받은 파일인 “python-3.12.0-amd64.exe”를 더블클릭하여 실행하면 나오는 Python 3.12 Setup 화면에서 "Add python.exe to PATH”와 “Use admin privileges when installing py.exe” 항목을 체크한 뒤 [Install Now] 버튼을 눌러 설치를 진행한다.



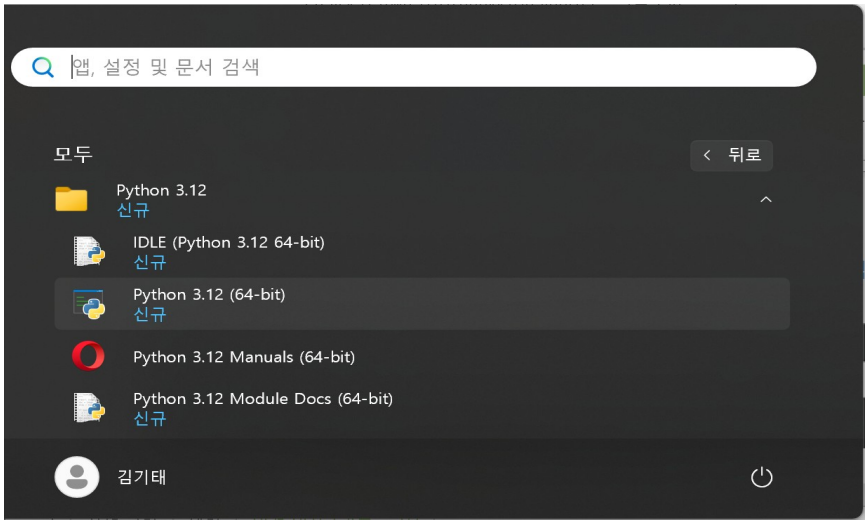
[그림 2] 파이썬 설치 시작 화면

③ Python 3.12 Setup 성공 화면에서 [Close] 버튼을 누르면 설치가 완료된다.



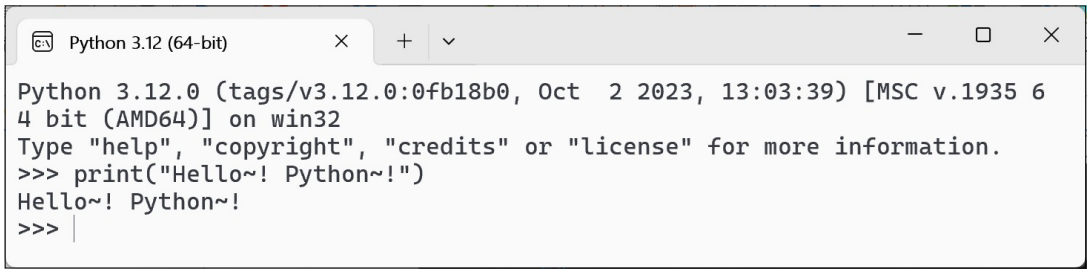
[그림 3] 파이썬 설치 성공 화면

④ 설치가 완료되면, 설치를 확인하기 위해 “모두”를 누르거나 윈도우즈 하단 웹 검색란에 “Python”입력하여 나오는 메뉴에서 “Python 3.12 (64-bit)”를 클릭하여 실행한다.



[그림 4] 파이썬 설치 시 프로그램 메뉴

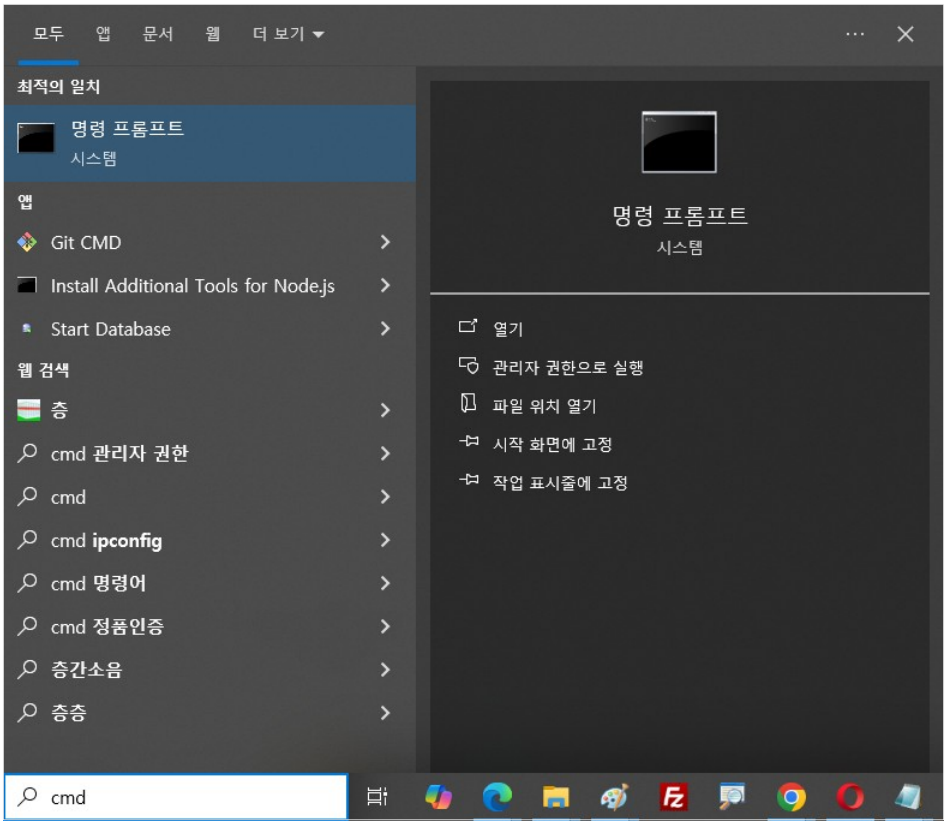
⑤ Python 스크립트 창이 실행되면 “>>>”의 뒤를 클릭한 후 print(”Hello~! Python~!”)을 입력하고, [Enter]키를 눌러 아래 화면과 같이 출력되는지 보고 제대로 파이썬이 실행되었는지 확인한다.



[그림 5] 파이썬 전용 터미널 확인

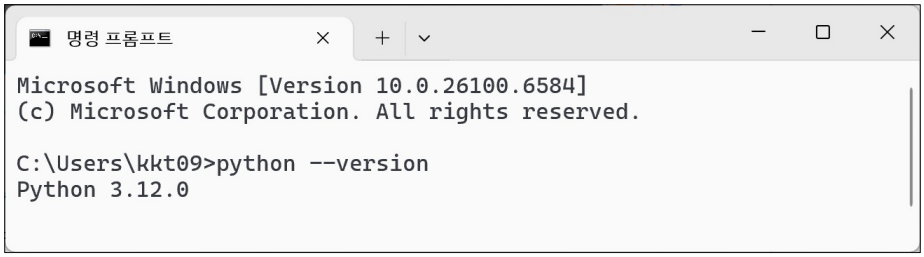
위 그림과 같이 파이썬 명령을 입력하였을 때 “Hello~! Python~!” 실행 결과가 나온다면, 일단 파이썬(Python) 설치는 무난하게 이상이 없이 설치된 것으로 볼 수 있다. 이렇듯 파이썬(Python) 스크립트는 파이썬(Python)의 간단한 구문들을 테스트해 볼 수 있으며, 이상유무 등도 확인할 수 있다.

⑥ 이번에는 Python 명령이 어디서든 실행되는지 파악하기 위해 “명령 프롬프트”인 “cmd”를 입력하여 아래 그림과 같이 실행한다. 명령 프롬프트는 명령을 실행하기 위한 터미널을 의미한다.



[그림 6] 명령 프롬프트(cmd) 실행

⑦ 명령 프롬프트가 실행되면, “python --version”을 입력하고 [Enter] 키를 눌러 버전이 출력되는지 확인한다. 아래 그림과 같이 버전이 제대로 출력되면, 어떤 디렉터리나 폴더의 위치에서도 파이썬이 실행되는 것이다.

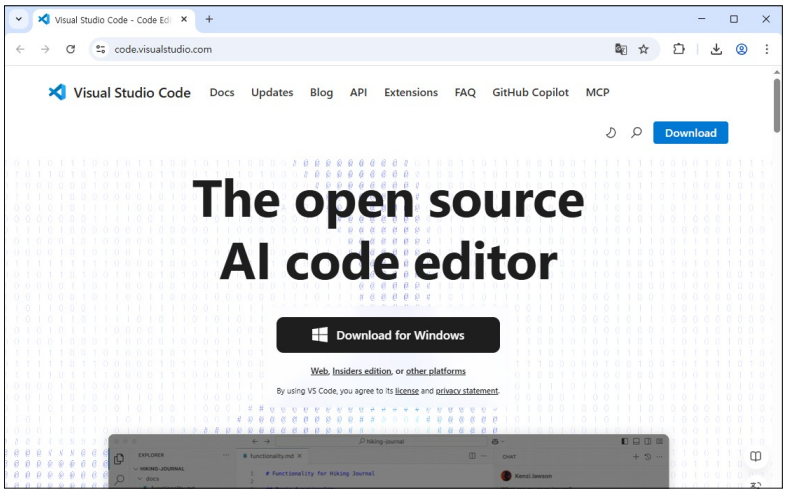


[그림 7] 파이썬 설치 확인

(2) VS Code 다운로드 및 설치

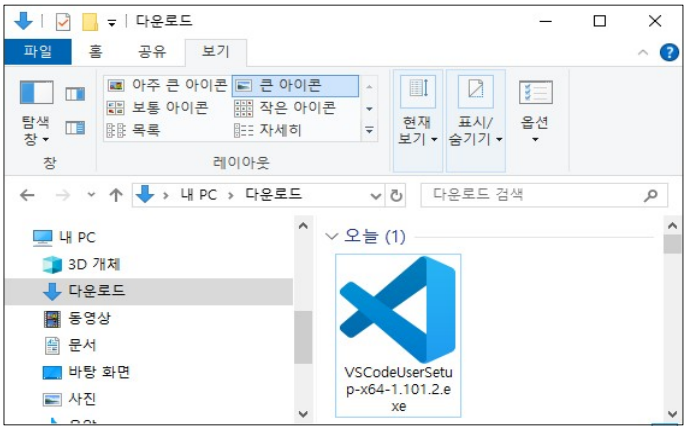
VS Code는 Python, SQL, Jupyter Notebook 등 다양한 언어를 통합적으로 지원하는 강력한 개발 도구입니다. PostgreSQL 확장과 Python 확장을 함께 설치하면 데이터 조회, 쿼리 실행, 모델 코드 작성까지 한 번에 처리할 수 있다.

① Visual Studio Code 공식 다운로드 사이트(https://code.visualstudio.com/)에 방문하여 [**Download for Windows**] 버튼을 눌러 VS Code를 다운로드한다.



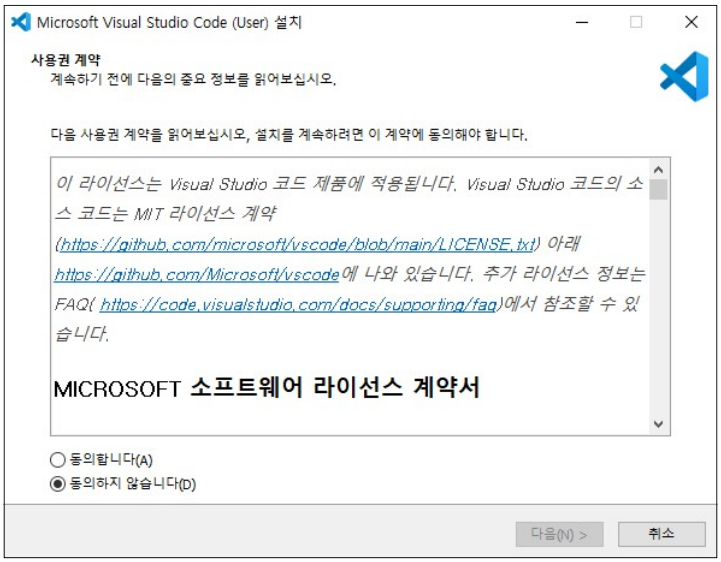
[그림 8] VS Code 다운로드 사이트

② 다운로드 폴더로 이동하여 다운로드가 된 “VSCodeUserSetup-x64-1.101.2.exe”를 더블클릭하여 설치를 시작한다.



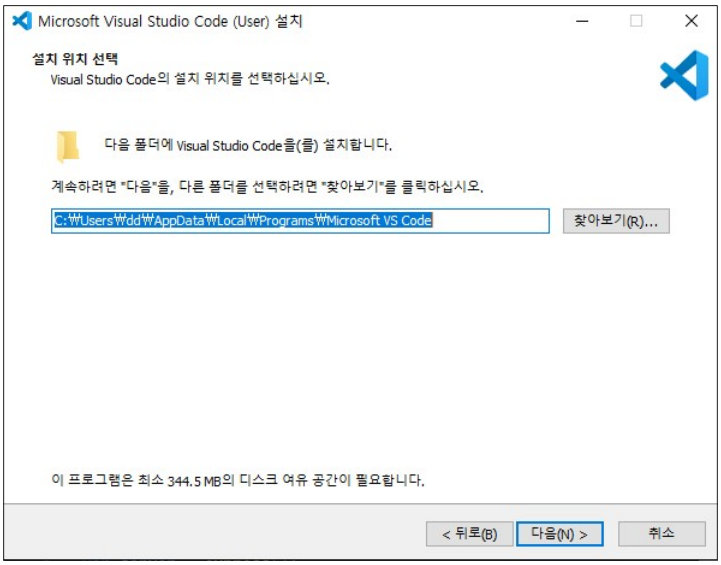
[그림 9] VS Code 설치 파일

③ Microsoft Visual Studio Code (User) 설치 시작화면이 로딩되면, “동의합니다”를 선택하고 [다음] 버튼을 누른다.



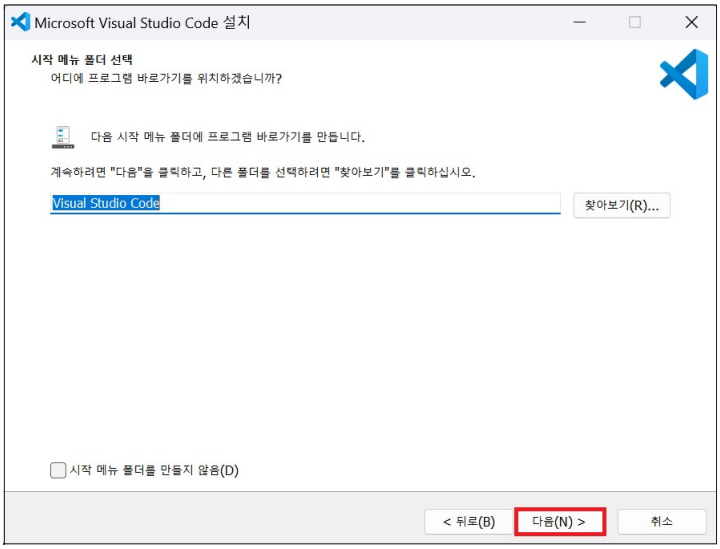
[그림 10] VS Code 설치 시 사용권 계약 화면

④ Microsoft Visual Studio Code (User) 설치 위치 선택화면이 로딩되면, 설치할 위치를 지정할 경우 “찾아보기(R)…” 버튼을 눌러 위치를 지정하고, 별도의 지정없이 기본값으로 설치할 경우는 [다음] 버튼을 누른다.



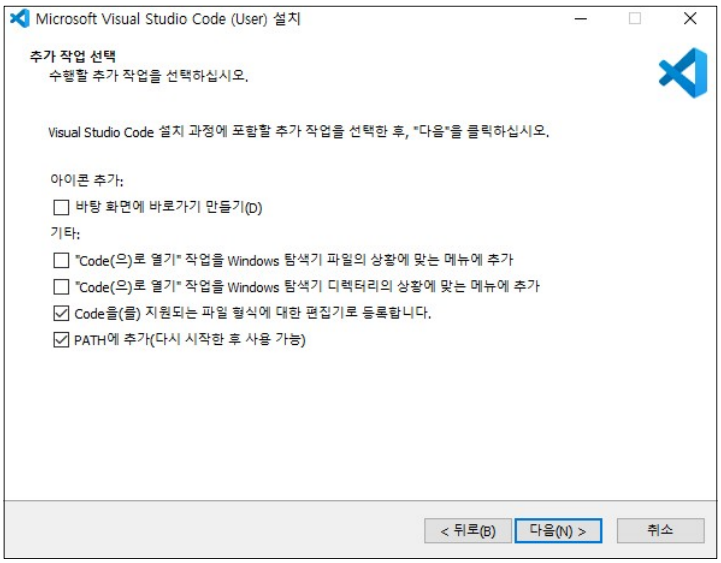
[그림 11] VS Code 설치 시 설치 위치 지정 화면

⑤ Microsoft Visual Studio Code (User) 시작 메뉴 폴더 선택화면이 로딩되면, 별도의 지정할 시작 메뉴 폴더가 없다면, [다음] 버튼을 누른다.



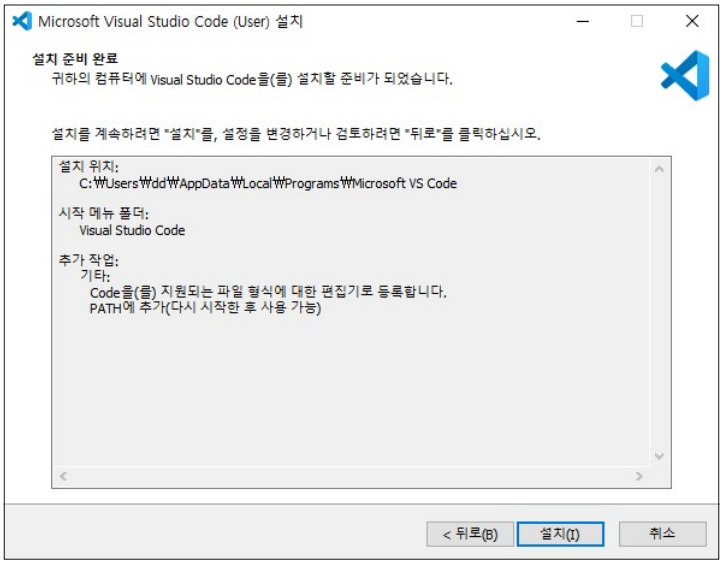
[그림 12] VS Code 설치 시 시작 메뉴 폴더 선택 화면

⑥ Microsoft Visual Studio Code (User) 추가 작업 선택 화면이 로딩되면, “Code을(를) 지원되는 파일 형식에 대한 편집기로 등록한다.” 항목과 “PATH에 추가(다시 시작한 후 사용 가능)”의 항목을 체크하고, [다음] 버튼을 누른다.



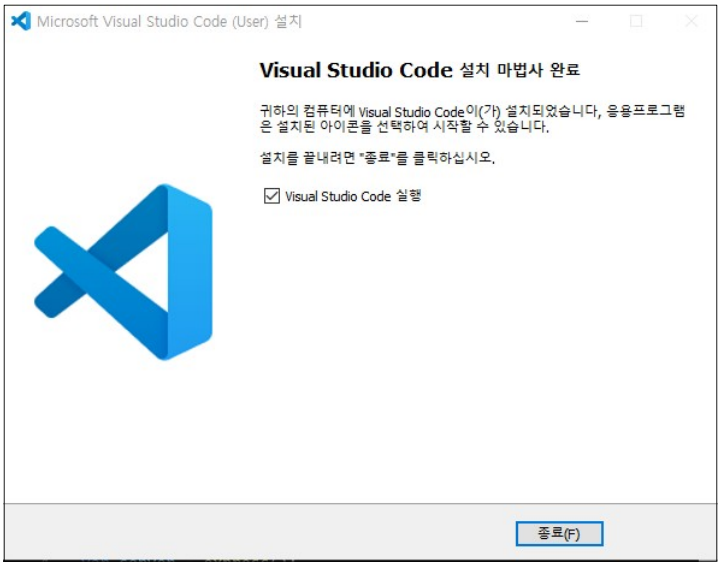
[그림 13] VS Code 설치 시 추가 작업 화면

⑦ Microsoft Visual Studio Code (User) 설치 준비 완료 화면이 로딩되면, [설치] 버튼을 눌러 설치를 진행한다.



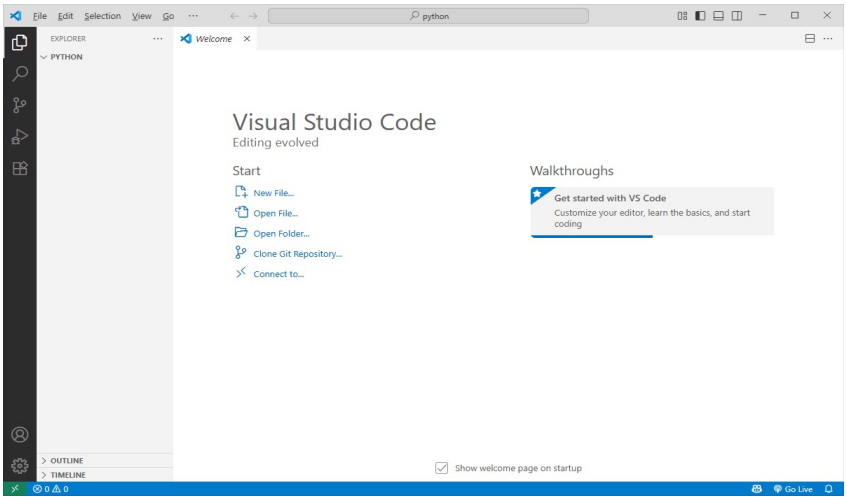
[그림 14] VS Code 설치 시 설치 준비 완료 화면

⑧ Microsoft Visual Studio Code (User) 설치 마법사 완료 화면이 로딩되면, [종료] 버튼을 눌러 설치를 마친다.



[그림 15] VS Code 설치 완료 화면

⑨ 설치가 종료되면, Visual Studio Code가 실행된다.

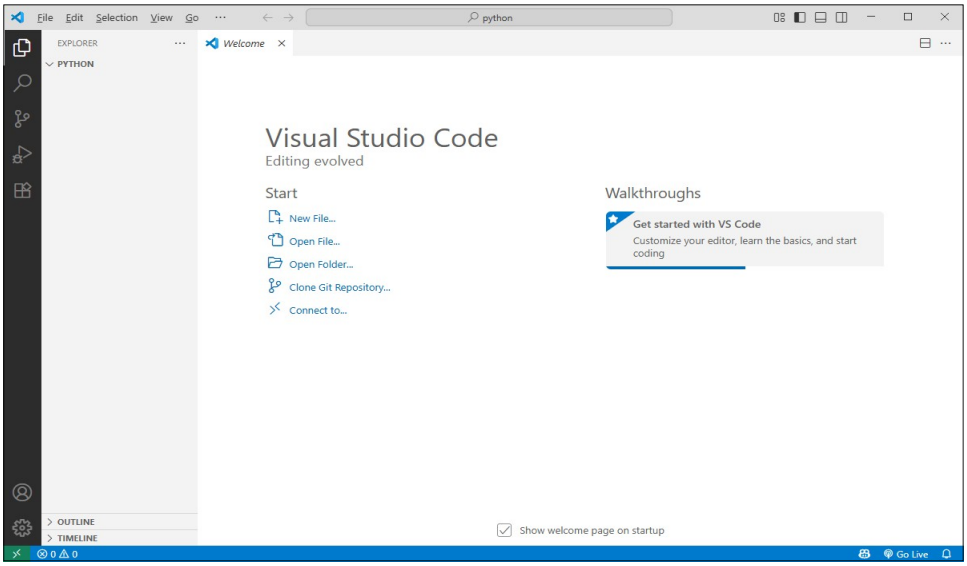


[그림 16] VS Code 성공 설치 시 자동 실행 화면

(3) VS Code 파이썬 개발 환경 설정

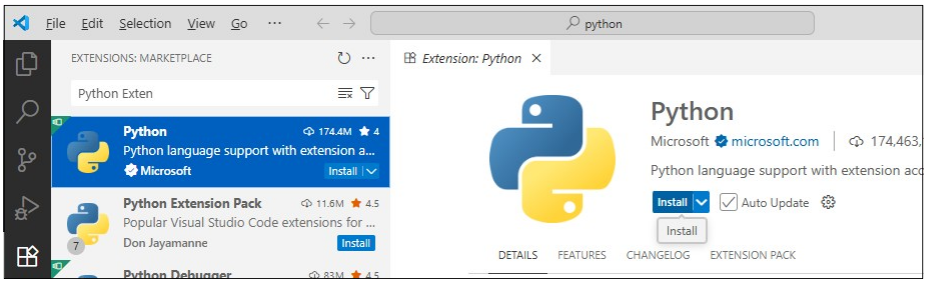
이번에는 VS Code에 파이썬 확장 기능을 설치하고, 가상환경 설정, 실행 및 디버깅 구성 등 개발에 필요한 핵심 환경을 체계적으로 준비하는 과정을 설명하며, 학습자가 편리하게 코드를 작성·실행·관리할 수 있도록 개발 환경을 완성하는 방법을 안내한다.

① Visual Studio Code(이하 VS Code)를 [시작] 메뉴 또는 [검색]을 통해 실행한다.



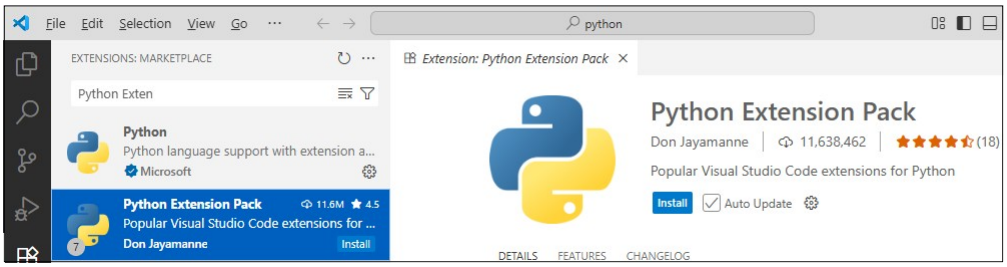
[그림 17] VS Code 실행 초기 화면

② 단축키 “Ctrl+Shift+X” 또는 좌측 툴바의 다섯 번째 아이콘인 “Extensions”를 눌러 확장(EXTENSIONS) 패널을 열고, “Python”을 검색하여 [Install] 버튼을 눌러 “Python” 확장 프로그램을 설치한다.



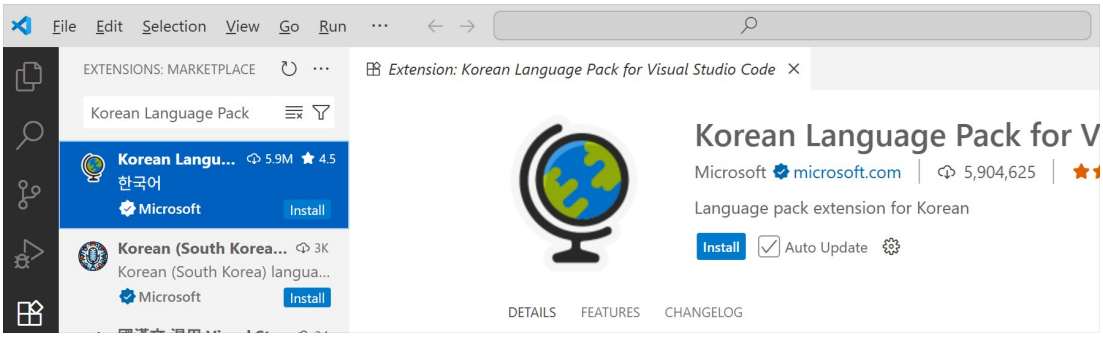
[그림 18] Python 확장 프로그램 설치 화면

③ 확장(EXTENSIONS) 패널에서 “Python Exctension Pack”을 검색하여 [Install] 버튼을 눌러 확장 프로그램을 설치한다.



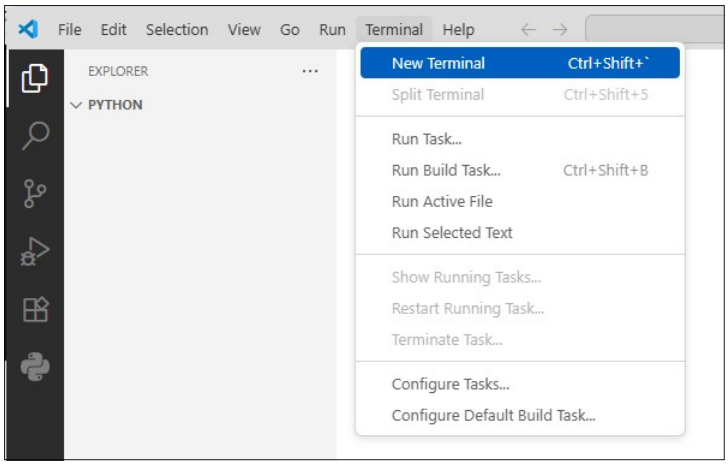
[그림 19] Python Extension Pack 설치 화면

③ 확장(EXTENSIONS) 패널에서 “Korean Language Pack”을 검색하여 [Install] 버튼을 눌러 확장 프로그램을 설치한다.



[그림 20] Korean Language Pack 설치 화면

⑤ [Terminal]-[New Terminal] 메뉴를 누르거나 단축키 “Ctrl+Shift+`”를 눌러 터미널을 실행한다.



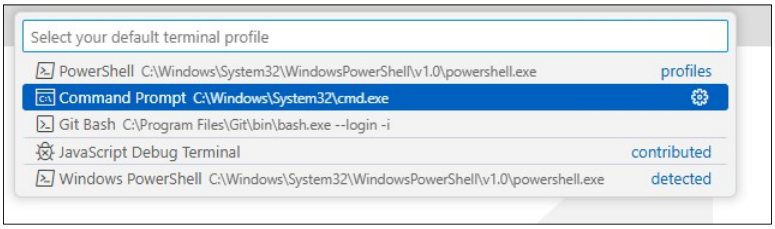
[그림 21] VS Code 터미널 실행

⑥ 터미널이 열리면, 기본 터미널인 PS(Power Shell)로 열린다. 우리는 “cmd” 터미널을 사용할 것이라서 기본 터미널을 “cmd” 터미널로 변경해야 한다. 그러므로, 아래 그림과 같이 터미널 오른쪽 상단의 [∨] 버튼을 눌러 나오면 메뉴 중에서 “Select Default Profile” 메뉴를 클릭하여 실행한다.



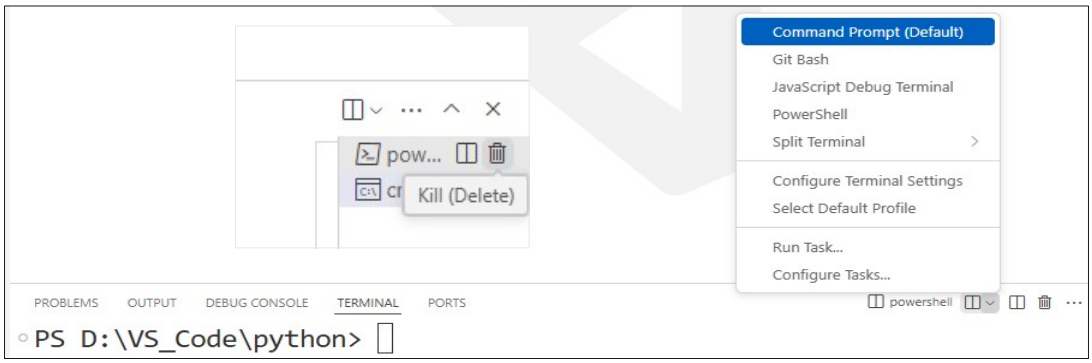
[그림 22] VS Code 기본 터미널 설정 메뉴

⑦ 커멘드 팔레트(Command Pallete)가 열리고, 나오는 목록에서 “Command Prompt”를 선택 후 [Enter] 키를 누른다.



[그림 23] VS Code 기본 터미널 선택 메뉴

⑧ 아래 그림과 같이 기본 터미널(Default Teminal)이 Command Prompt로 변경되는 것을 알 수 있으며, 나오는 메뉴에서 “Command Prompt (Default)를 누르고, 오른쪽 터미널 목록에서 “powershell”은 휴지통 아이콘을 눌러 삭제한다.



[그림 24] VS Code 기본 터미널 변경 완료(오른쪽) 및 삭제(왼쪽)

📌 TIP | VS Code의 터미널

Command Prompt (CMD)
- 윈도우의 기본 명령줄 인터프리터이다.
- 오래된 MS-DOS 기반 명령어 체계를 사용한다.
- 윈도우 환경에서 가장 호환성이 높지만 기능은 단순하다.

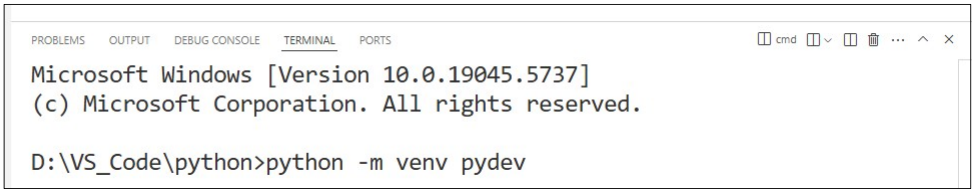
PowerShell
- CMD보다 진보된 윈도우용 셸로,
- 객체(Object) 기반 파이프라인을 지원합니다.
- 윈도우뿐 아니라 macOS/Linux에도 설치 가능한 범용 셸입니다.

Git Bash
- Git 설치 시 함께 제공되는 Bash 셸 환경입니다.
- 리눅스/유닉스 명령어(ls, cat, grep, rm, chmod 등)를
- Windows에서 그대로 사용 가능하게 만들어 줍니다.
- 실제로는 MinGW(Minimal GNU for Windows) 기반의 Bash입니다.

(4) 가상환경(venv) 생성

AI 프로젝트에서는 독립된 가상환경을 구성해야 패키지 충돌을 방지하고 프로젝트별 버전 일관성을 유지할 수 있습니다. venv를 사용해 환경을 생성하고 필수 라이브러리를 설치한다.

① cmd(Command Prompt) 터미널에서 “python -m venv pydev”를 입력하여 “pydev”라는 이름의 파이썬 가상 환경을 만든다. 가상 환경을 만들면, 현재 위치에 가상 환경 이름으로 디렉터리가 생성되며, 그 안에 필요한 라이브러리와 실행 파일을 복사하여 저장되는 것을 알 수 있다.



[그림 25] 파이썬 가상 환경 생성 화면

② 이제 가상 환경을 활성화하여 프로젝트를 진행해야 되겠다. 가상 환경을 활성화하려면, 가상 환경 이름으로 된 디렉터리 안의 “Scripts/activate.bat”를 실행해야 한다.
그러기 위해서 “pydev₩Scripts₩activate”를 입력하고, [Enter]키를 누른다. 이 때 형식은 “가상환경 이름₩Scripts₩activate”와 같다.



[그림 26] 파이썬 가상 환경 활성화 화면

Python 가상환경 관련 명령

구분	명령어	설명	비고
가상환경 생성	python -m venv .venv	현재 디렉터리에 .venv 폴더 생성	표준 내장 방식
가상환경 생성(virtualenv)	virtualenv .venv	virtualenv 패키지로 생성	pip install virtualenv 필요
가상환경 활성화	.venv\Scripts\activate	커맨드 프롬프트용	(venv) 프롬프트 표시 확인
가상환경 비활성화	deactivate	현재 가상환경 종료	모든 OS 공통
가상환경 위치 확인	which python 또는 where python	현재 파이썬 실행 경로 출력	venv 경로 확인용
패키지 설치	pip install <package>	가상환경 내 패키지 설치	전역 환경과 분리됨
패키지 목록 저장	pip freeze > requirements.txt	현재 설치된 패키지 버전 기록	배포 시 필수
패키지 복원	pip install -r requirements.txt	동일한 환경 재현	CI/CD 환경에서 활용
가상환경 삭제	rm -rf .venv	폴더 삭제로 제거	Windows는 rmdir /s .venv
파이썬 버전 지정 생성	python3.12 -m venv .venv	특정 버전으로 환경 생성	다중 파이썬 버전 관리 시 유용

(5) 의존성 관리

이 절에서는 프로젝트별 의존성 관리 전략을 비교하고, Python 표준 venv 기반에서 pip로 관리하는 방법과 현대적 툴인 Poetry를 사용한 의존성 선언·잠금(lock)·패키징 절차를 모두 다룬다. 또한 requirements.txt·pyproject.toml·lock 파일의 역할, 개발/운영(Dev/Prod) 의존성 구분, CI/CD 및 컨테이너 빌드 시 권장 명령어와 보안·재현성(Deterministic) 확보 방법까지 실습 예제와 함께 제시한다.

① 패키지 설치 (pip)

가장 기본적이고 보편적인 방식은 Python 내장 venv로 가상환경을 만들고, pip로 패키지를 설치해 requirements.txt로 의존성을 기록·복원하는 것으로, 소규모 프로젝트나 단순한 배포에 적합하다.
그럼 이제, 가상환경 내 pip install로 필요한 패키지를 설치한다.

```
pip install fastapi uvicorn[standard] sqlalchemy asyncpg
```

➡ uvicorn[standard]처럼 extras를 지정해 추가 의존성을 함께 설치할 수 있다.

② 의존성 내보내기(requirements.txt)

애플리케이션의 이전을 위해 "requirements.txt"에 의존성 라이브러리를 기록한다. 이때 현재 가상환경의 정확한 패키지 목록과 버전을 기록하는 명령은 pip freeze로 한다.

```
pip freeze > requirements.txt
```

➡ requirements.txt는 재현 가능한 설치를 위해 정확한 버전(예: fastapi==0.110.0)을 포함한다.

③ 의존성 설치 및 복원

애플리케이션의 이전은 의존성 라이브러리를 설치해야 한다. 다른 환경이나 서버에서 requirements.txt로 동일한 환경을 복원한다.

```
pip install -r requirements.txt
```

➡ CI나 컨테이너 빌드 시 이 명령을 사용해 동일한 패키지 집합을 재현한다.

④ 운영 환경에서의 권장 옵션

이미지 빌드 시 캐시 최소화와 보안 강화를 위해 --no-cache-dir와 --disable-pip-version-check 옵션을 사용한다.

```
RUN pip install --no-cache-dir --disable-pip-version-check -r requirements.txt
```

➡ 이미지 크기와 빌드 변동성을 줄여 일관된 프로덕션 이미지를 만드는 데 유리하다.

⑤ pip-tools 설치 및 개념

직접 pip freeze로 관리하는 방식은 최상위 의존성만 바꿔도 하위 의존성이 불일치할 수 있어 pip-tools 같은 도구로 상위(requirements.in)와 고정된 최종(requirements.txt) 파일을 분리 관리하는 것이 권장된다. pip-tools의 pip-compile은 상위 의존성만 명시한 requirements.in에서 재현 가능한

requirements.txt를 생성한다.

```
pip install pip-tools
```

➡ 상위(declared)와 하위(resolved) 의존성을 분리해 관리하면 업데이트·검토가 쉬워진다.

⑥ pip-tools의 requirements.in 기본

상위 의존성 파일에는 최소한의 요구만 적는다.

```
fastapi
uvicorn[standard]
sqlalchemy
```

➡ 버전을 고정하지 않으면 pip-compile이 최신 호환 버전으로 하위 의존성을 잠금한다.

⑦ pip-compile로 잠금 파일 생성

pip-compile을 실행하면 requirements.txt가 생성된다.

```
pip-compile requirements.in
```

➡ 생성된 requirements.txt에는 모든 종속 패키지와 정확한 버전이 포함된다.

⑧ pip-sync로 환경 동기화

pip-sync는 requirements.txt와 환경을 일치시켜 불필요한 패키지를 제거한다.

```
pip-sync requirements.txt
```

➡ 개발 중에 설치된 임시 패키지를 정리해 환경을 깨끗하게 유지한다.

⑨ 현대적 의존성·패키지 관리를 위한 Poetry

Poetry는 pyproject.toml을 중심으로 의존성 선언, 잠금, 패키징, 가상환경 관리를 통합 제공하며, 프로젝트 생성부터 배포까지 일관된 경험을 제공하므로 중·대형 프로젝트에 적합하다. 그럼 Poetry를 설치 후 poetry init으로 프로젝트를 시작하고 pyproject.toml 파일을 생성해보자

```
curl -sSL https://install.python-poetry.org | python3 -
poetry new myproj     # 또는 poetry init
```

➡ 설치 스크립트는 사용자 홈에 Poetry를 설치하고 poetry CLI를 사용 가능하게 한다.

⑩ Poetry를 활용한 의존성 추가

poetry add로 런타임 의존성, poetry add --dev로 개발 의존성을 분리해 관리한다.

```
poetry add fastapi uvicorn[standard] sqlalchemy asyncpg
poetry add --dev pytest black
```

➡ pyproject.toml과 poetry.lock이 생성되어 재현 가능한 설치가 보장된다.

⑪ Poetry를 활용한 가상환경 관리 방식

Poetry는 프로젝트별 가상환경을 자동 생성하거나 시스템 인터프리터를 사용하도록 설정할 수 있다.

```
poetry shell       # poetry가 관리하는 venv로 진입
poetry env info    # 가상환경 위치 확인
```

➡ CI에서는 poetry install --no-root로 의존성만 설치하는 패턴을 쓴다.

⑫ Poetry를 활용한 잠금(lock) 파일과 재현성

poetry.lock에는 정확한 버전과 해시가 들어 있어 다른 환경에서 동일한 의존성을 보장한다.

```
poetry lock
poetry install --no-interaction
```

➡ poetry install은 pyproject.toml과 poetry.lock을 참고해 동일한 환경을 구성한다.

⑬ Poetry를 활용한 배포 및 패키징

poetry build 및 poetry publish로 패키지화해 PyPI나 사설 레지스트리에 업로드할 수 있다.

```
poetry build
poetry publish --username <user> --password <pw>
```

➡ Poetry는 배포 메타데이터와 의존성 관리를 일관되게 처리하므로 라이브러리 개발에도 유리하다.

pip 필수 핵심 명령

목적	명령어 예시
pip 최신화	`python -m pip install --upgrade pip`
패키지 설치	`pip install requests`
버전 지정 설치	`pip install fastapi==0.115.0`
여러 패키지 한 번에	`pip install -r requirements.txt`
설치 목록 저장	`pip freeze > requirements.txt`
requirements 재설치	`pip install -r requirements.txt`
업데이트 가능한 목록 확인	`pip list --outdated`
캐시 초기화	`pip cache purge`
pip 설정 변경	`pip config set global.index-url https://pypi.org/simple`

> 📌 **TIP | Python 패키지 관리 실무 활용**
>
> ♨ "pip은 간단하게, pip-tools는 안전하게, Poetry는 체계적으로."하며, 규모와 목적에 맞는 관리 방식을 선택하면 된다.
>
> **pip**: 기본 패키지 관리자로 MSA와 같은 소규모 프로젝트에 적합하며, 단순하고 빠르다.
> **pip-tools**: pip의 확장판으로 모놀리식 방식의 중규모 프로젝트에 적합하며, 안정적이다.
> **Poetry**: 패키지를 통합 관리할 수 있으며, 대규모 프로젝트에 적합하고, 선언, 잠금, 빌드를 일체화 할 수 있다.